

## Lab Final Report

Course Code: CSE-406

Course Title: Digital Image Processing Laboratory  
4<sup>th</sup> Year 1<sup>st</sup> Semester Examination 2023

Date of Submission: 10-12-2024



Submitted to-

**Dr. Morium Akter**

*Professor*

**Dr. Md. Golam Moazzem**

*Professor*

*Department of Computer Science and Engineering  
Jahangirnagar University  
Savar, Dhaka-1342*

| Sl | Class Roll | Exam Roll | Name            |
|----|------------|-----------|-----------------|
| 01 | 364        | 202176    | Suraiya Mahmuda |

# Experiment Name: Take an image as input and then display/print that image

## Objectives of the Program

- **Load an Image:** The program loads an image from a specified file path.
- **Display the Image:** It displays the image so the user can see the original content.
- **Demonstrate Image Processing:** The program showcases a simple image processing task (converting the image to grayscale).
- **Foundation for Further Processing:** The program sets up a framework that can be expanded with more advanced image processing techniques, such as filtering, edge detection, or object recognition.

## Source Code:

```
% Specify the filename of the image
filename = 'cat.jpg'; % Replace with your image file name and extension

% Check if the file exists in the current directory or provide the full path
if exist(filename, 'file') == 2
    % Read the image file
    img = imread(filename);

    % Display the image
    imshow(img);
    title('Displayed Image'); % Add a title to the image
else
    % Display an error message if the file does not exist
    error('File does not exist. Please check the filename and path.');
```

## Input:



shutterstock.com · 2462995331

## Output:



shutterstock.com · 2462995331

## Experiment Name: Take an image as input and display it's negative image.

### Objectives:

- **Read the Input Image:** Load the image into MATLAB.
- **Convert the Image to Negative:** Transform the pixel values to their negative counterparts.
- **Display the Original and Negative Images:** Show the images for comparison.

### Source Code:

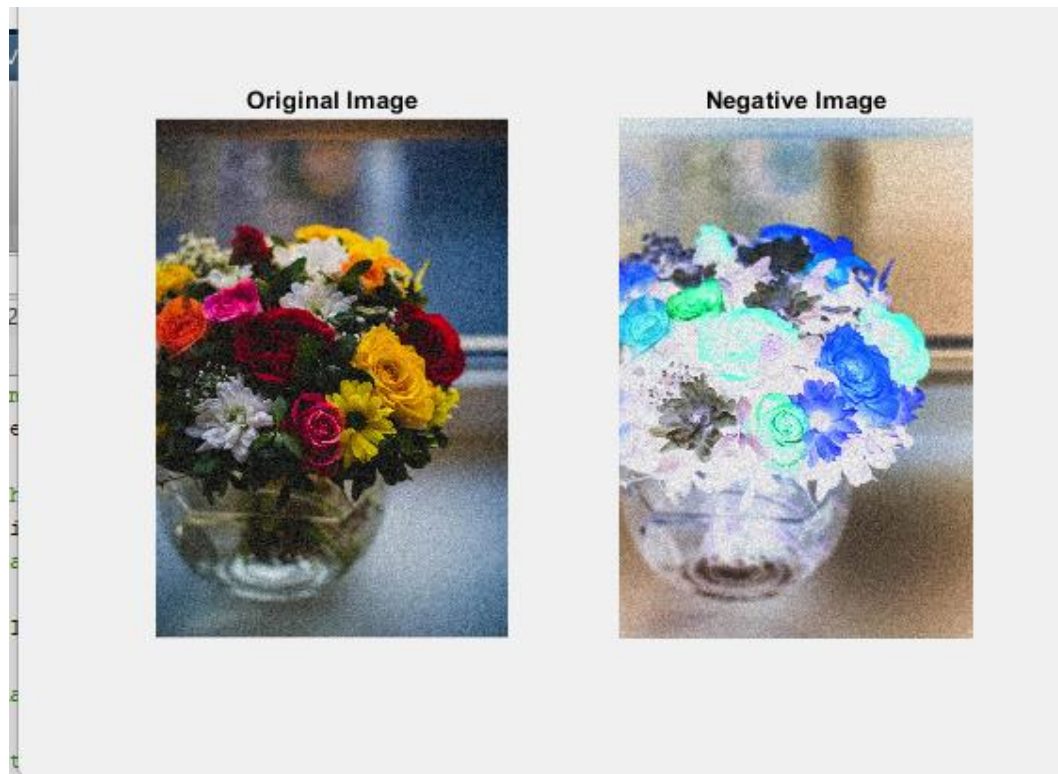
```
% Read the image
originalImage = imread('C:\Users\CSELAB-2\Desktop\GIPLab02\negativeimage1.jpg');

% Check if the image is grayscale or RGB
if size(originalImage, 3) == 1
    % Grayscale image
    maxValue = 255; % Maximum value for 8-bit images
    negativeImage = maxValue - originalImage;
else
    % RGB image
    maxValue = 255; % Maximum value for 8-bit images
    % Apply the negative transformation to each channel
    negativeImage = maxValue - originalImage;
end

% Display the original and negative images
figure;
subplot(1, 2, 1);
imshow(originalImage);
title('Original Image');

subplot(1, 2, 2);
imshow(negativeImage);
title('Negative Image');
```

### Input and Output:



## Experiment Name: Take an image as input and enhance it's contrast stretching image.

### Objectives

1. **Read the Input Image:** Load the image into MATLAB.
2. **Define the Contrast Stretching Parameters:** Determine the minimum and maximum values for both the original and stretched image.
3. **Apply Contrast Stretching:** Transform the pixel values based on the defined parameters.
4. **Display the Original and Enhanced Images:** Show both the original and contrast-stretched images for comparison.

### Source Code:

```
% Step 1: Read the RGB image
img = imread('C:\Users\CSELAB-2\Desktop\GIPLab02\contast.jpg'); % Replace
'your_image.jpg' with your image filename

% Step 2: Define the stretching parameters
min_out = 0;           % Minimum intensity of the output image
max_out = 255;         % Maximum intensity of the output image

% Initialize the stretched image
img_stretched = img;

% Process each channel separately
for c = 1:size(img, 3)
    % Get the current channel
    channel = img(:,:,c);

    % Define the input range for the current channel
```

```

min_in = double(min(channel(:)));
max_in = double(max(channel(:)));

% Apply contrast stretching
channel_double = double(channel);
channel_stretched = ((channel_double - min_in) / (max_in - min_in)) * (max_out - min_out) + min_out;

% Store the result in the output image
img_stretched(:,:,c) = uint8(channel_stretched);
end

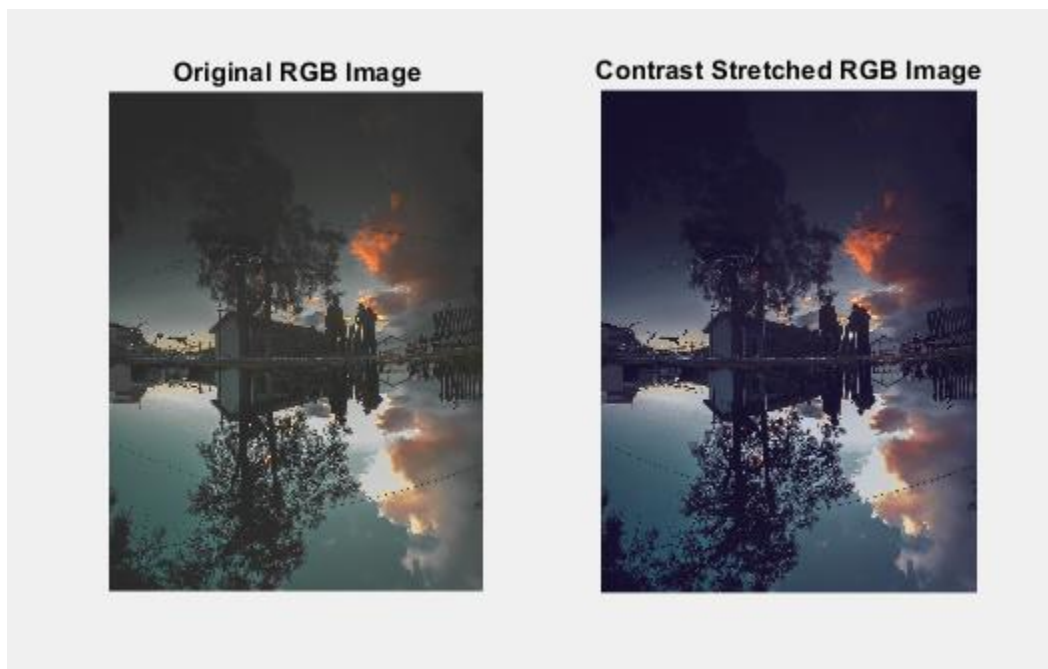
% Step 3: Display the original and stretched images
figure; % Open a new figure window

subplot(1,2,1);
imshow(img);
title('Original RGB Image');

subplot(1,2,2);
imshow(img_stretched);
title('Contrast Stretched RGB Image');

```

### Input and Output:



### Experiment Name:

- Histogram equalization of a given image to show the histogram of original image and equalized image.
- Cumulative Distribution Function(CDF) graph of pixel intensity of original vs equalized image.
- Histogram specification using Bimodal Gaussian Function to show the histogram of original and modified image.

### a. Objectives:

- Display the original image.
- Compute the histogram of the original image.
- Perform histogram equalization to improve contrast.
- Display the equalized image.
- Compute the histogram of the equalized image.

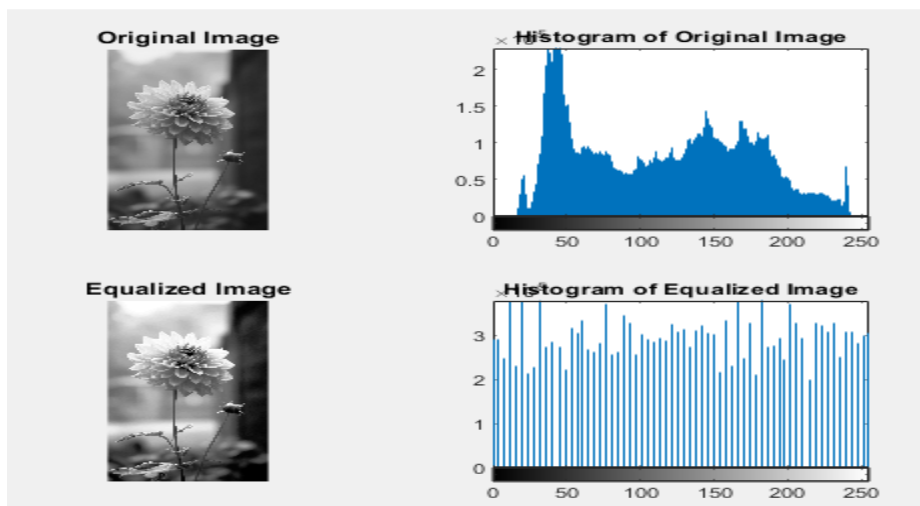
#### Source Code:

```
% Read the input image
input_image = imread('C:\Users\CSELAB-2\Desktop\New folder\beautiful-flower-nature.jpg'); %
Replace 'example.jpg' with your image file
% Convert to grayscale if it's a color image
if size(input_image, 3) == 3
    input_image = rgb2gray(input_image);
end
% Display the original image
figure;
subplot(2, 2, 1);
imshow(input_image);
title('Original Image');
% Compute and display the histogram of the original image
subplot(2, 2, 2);
imhist(input_image);
title('Histogram of Original Image');
% Perform histogram equalization
equalized_image = histeq(input_image);
% Display the equalized image
subplot(2, 2, 3);
imshow(equalized_image);
title('Equalized Image');
% Compute and display the histogram of the equalized image
subplot(2, 2, 4);
imhist(equalized_image);
title('Histogram of Equalized Image');
% Optional: Save the results
imwrite(equalized_image, 'equalized_image.jpg'); % Save the equalized image
```

#### Input:



**Output:**



## b. Objectives:

- Compute and plot the CDF of the original image.
- Perform histogram equalization.
  - Compute and plot the CDF of the equalized image.

**Source Code:**

```
% Define the input image and transformation parameters
```

```

inputImageFile = 'C:\Users\CSELAB-2\Desktop\New folder\beautiful-flower-nature.jpg'; % Replace
with your image file
% Read the image
img = imread(inputImageFile);
% Convert to grayscale if the image is RGB
if size(img, 3) == 3
    img = rgb2gray(img);
end
% Convert image to double for processing
img = double(img) / 255; % Normalize to [0, 1]
% Define the linear transformation parameters
a = 1.5; % Scaling factor
b = 0.2; % Offset
% Apply the linear transformation
transformed_img = a * img + b;
% Clip values to [0, 1] range
transformed_img = min(max(transformed_img, 0), 1);
% Convert back to uint8
transformed_img = uint8(255 * transformed_img);
% Display the original and transformed images
figure;
subplot(1, 2, 1);
imshow(img);
title('Original Image');
subplot(1, 2, 2);
imshow(transformed_img);
title('Transformed Image');

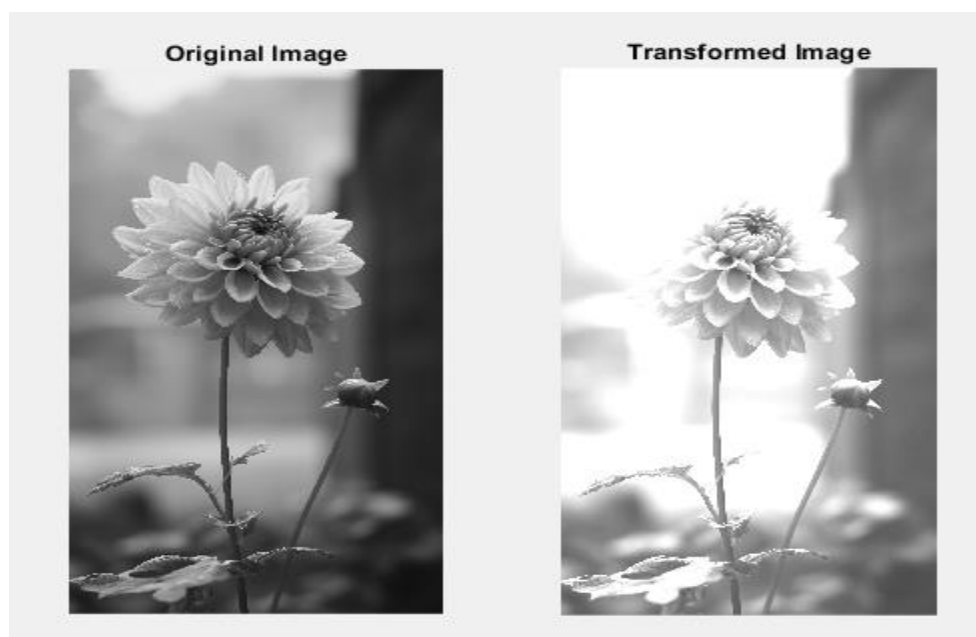
```

**Input:**





**Output:**



### C. Objectives:

- Display the original image and its histogram.
- Define a target histogram based on a bimodal Gaussian function.
- Perform histogram specification (matching) to make the original image's histogram resemble the target bimodal distribution.
- Display the modified image and its histogram.

### Source Code:

```
% Read the source and target images
I_source = imread('C:\Users\CSELAB-2\Desktop\New folder\Source.jpg'); % Replace with your
source image file
I_target = imread('C:\Users\CSELAB-2\Desktop\New folder\Target.jpg'); % Replace with your
target image file
% Convert images to grayscale if needed
I_source_gray = rgb2gray(I_source);
I_target_gray = rgb2gray(I_target);
% Compute histograms
[counts_source, gray_levels_source] = imhist(I_source_gray);
[counts_target, gray_levels_target] = imhist(I_target_gray);
% Compute cumulative histograms
cdf_source = cumsum(counts_source) / numel(I_source_gray);
cdf_target = cumsum(counts_target) / numel(I_target_gray);
% Create an empty mapping array
mapping = zeros(256, 1);
% Mapping source CDF to target CDF
for i = 1:256
    [~, idx] = min(abs(cdf_source(i) - cdf_target));
    mapping(i) = gray_levels_target(idx);
end
% Apply the mapping to the source image
I_source_mapped = uint8(arrayfun(@(x) mapping(x + 1), I_source_gray));
% Display the results
figure;
subplot(1, 3, 1);
imshow(I_source_gray);
title('Original Source Image');
subplot(1, 3, 2);
imshow(I_target_gray);
title('Target Image');
subplot(1, 3, 3);
imshow(I_source_mapped);
title('Histogram-Matched Image');
```

### Input:



**Output:**



**Experiment Name:**

- a. Image filtering using Mean, Median and Gaussian techniques.
- b. Image resizing using Replication method, Linear Interpolation method and Pixel Skipping method.

**a. Objectives:**

**Mean Filter:**

- Objective: Reduce overall noise by averaging pixel values, but it may blur edges due to uniform smoothing.

**Median Filter:**

- Objective: Remove impulse noise (salt-and-pepper noise) while preserving edges and avoiding blurring.

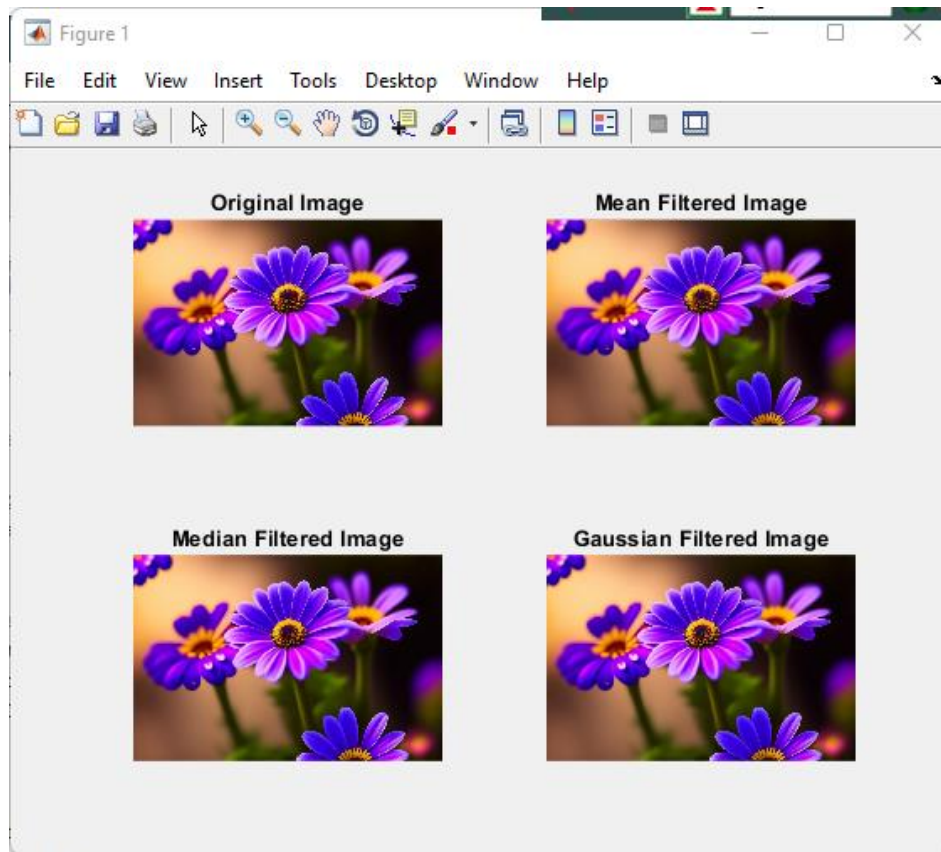
#### Gaussian Filter:

- Objective: Achieve a smooth image by using a Gaussian function to weigh neighboring pixels, providing more natural blurring and reducing Gaussian noise.

#### Source Code:

```
% Read the color image
colorImage = imread('C:\Users\CSELAB-2\Desktop\Lab04\Filters.jpg');
% Separate the color channels
redChannel = colorImage(:,:,1);
greenChannel = colorImage(:,:,2);
blueChannel = colorImage(:,:,3);
% Mean Filter
meanFilter = fspecial('average', [3 3]);
meanFilteredRed = imfilter(redChannel, meanFilter);
meanFilteredGreen = imfilter(greenChannel, meanFilter);
meanFilteredBlue = imfilter(blueChannel, meanFilter);
meanFilteredImage = cat(3, meanFilteredRed, meanFilteredGreen, meanFilteredBlue);
% Median Filter
medianFilteredRed = medfilt2(redChannel, [3 3]);
medianFilteredGreen = medfilt2(greenChannel, [3 3]);
medianFilteredBlue = medfilt2(blueChannel, [3 3]);
medianFilteredImage = cat(3, medianFilteredRed, medianFilteredGreen, medianFilteredBlue);
% Gaussian Filter
gaussianFilter = fspecial('gaussian', [3 3], 0.5);
gaussianFilteredRed = imfilter(redChannel, gaussianFilter);
gaussianFilteredGreen = imfilter(greenChannel, gaussianFilter);
gaussianFilteredBlue = imfilter(blueChannel, gaussianFilter);
gaussianFilteredImage = cat(3, gaussianFilteredRed, gaussianFilteredGreen, gaussianFilteredBlue);
% Display the original and filtered images
figure;
subplot(2,2,1), imshow(colorImage), title('Original Image');
subplot(2,2,2), imshow(meanFilteredImage), title('Mean Filtered Image');
subplot(2,2,3), imshow(medianFilteredImage), title('Median Filtered Image');
subplot(2,2,4), imshow(gaussianFilteredImage), title('Gaussian Filtered Image');
```

#### Input and Output:



## b. Objectives:

**Replication:** Retain simplicity and speed but risk blockiness.

**Linear Interpolation:** Preserve smoothness and visual quality with balanced performance.

**Pixel Skipping:** Maximize speed in downsizing but risk losing detail and introducing aliasing artifacts.

## Source Code:

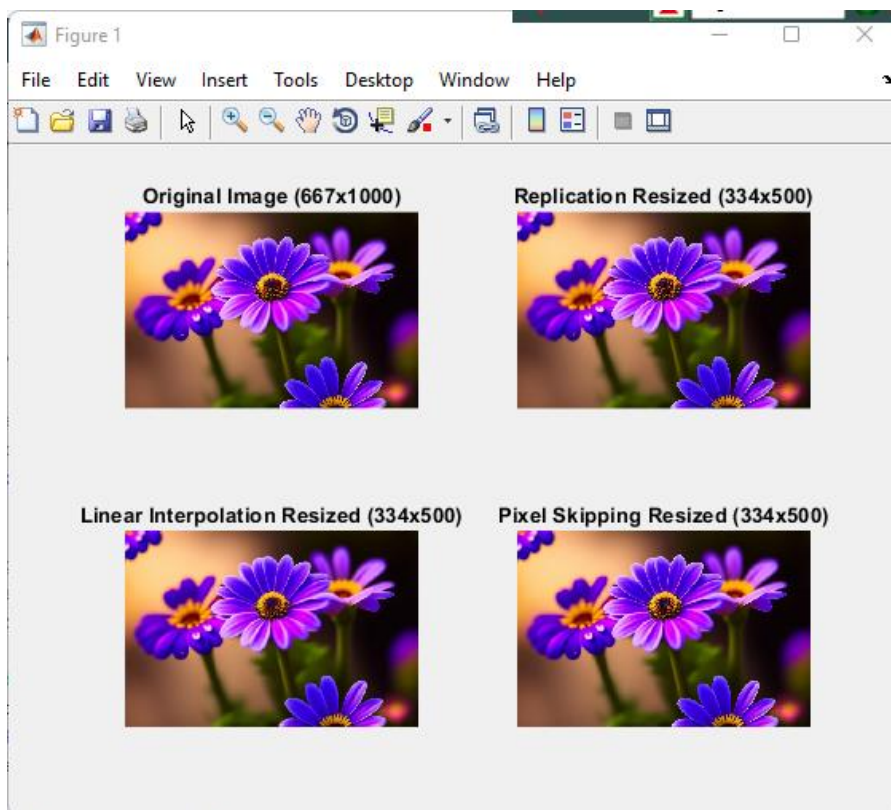
```
% Read the color image
colorImage = imread('C:\Users\CSELAB-2\Desktop\Lab04\Filters.jpg');
% Display original image size
originalSize = size(colorImage);
disp('Original Image Size:');
disp(originalSize);
% Resize the image using nearest-neighbor interpolation
scaleFactor = 0.5; % Example scale factor for downscaling
nearestNeighborResizedImage = imresize(colorImage, scaleFactor, 'nearest');
% Display nearest-neighbor resized image size
nearestNeighborSize = size(nearestNeighborResizedImage);
disp('Nearest-Neighbor Resized Image Size:');
disp(nearestNeighborSize);
% Resize the image using linear interpolation
linearResizedImage = imresize(colorImage, scaleFactor, 'bilinear');
% Display linear interpolation resized image size
```

```

linearSize = size(linearResizedImage);
disp('Linear Interpolation Resized Image Size:');
disp(linearSize);
% Resize the image using pixel skipping method
skipFactor = 1.0/scaleFactor; % Example skip factor
pixelSkippedImage = colorImage(1:skipFactor:end, 1:skipFactor:end, :);
% Display pixel skipping resized image size
pixelSkippedSize = size(pixelSkippedImage);
disp('Pixel Skipping Resized Image Size:');
disp(pixelSkippedSize);
% Display the original and resized images with dimensions
figure;
subplot(2,2,1), imshow(colorImage), title(['Original Image (' num2str(originalSize(1)) 'x'
num2str(originalSize(2)) ')']);
subplot(2,2,2), imshow(nearestNeighborResizedImage), title(['Replication Resized ('
num2str(nearestNeighborSize(1)) 'x' num2str(nearestNeighborSize(2)) ')']);
subplot(2,2,3), imshow(linearResizedImage), title(['Linear Interpolation Resized ('
num2str(linearSize(1)) 'x' num2str(linearSize(2)) ')']);
subplot(2,2,4), imshow(pixelSkippedImage), title(['Pixel Skipping Resized ('
num2str(pixelSkippedSize(1)) 'x' num2str(pixelSkippedSize(2)) ')']);

```

### Input and Output:





# Experiment Name: Edge Detection using Prewitt operator, Roberts operator, Sobel operator and Isotropic operator

## Objectives:

**Prewitt operator:** Detect edges by calculating the gradient of the image intensity at each pixel using a simple approximation of the derivative.

**Roberts operator:** Detect edges by calculating the approximate gradient of the image using a 2x2 matrix, which highlights diagonal edges.

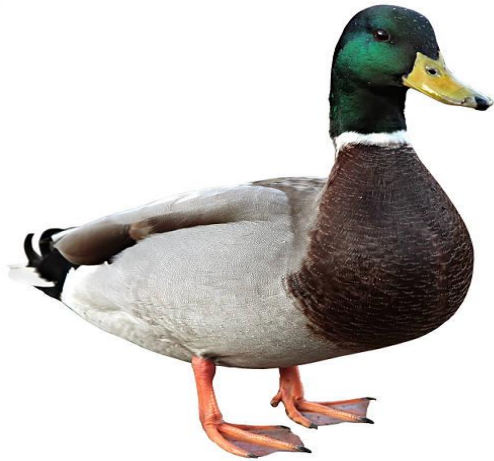
**Sobel operator:** Detect edges by calculating the gradient magnitude and direction using a larger convolution kernel, which offers more robustness to noise.

**Isotropic operator:** Detect edges in an image while being rotation-invariant (i.e., it detects edges irrespective of their direction), using second-order derivatives.

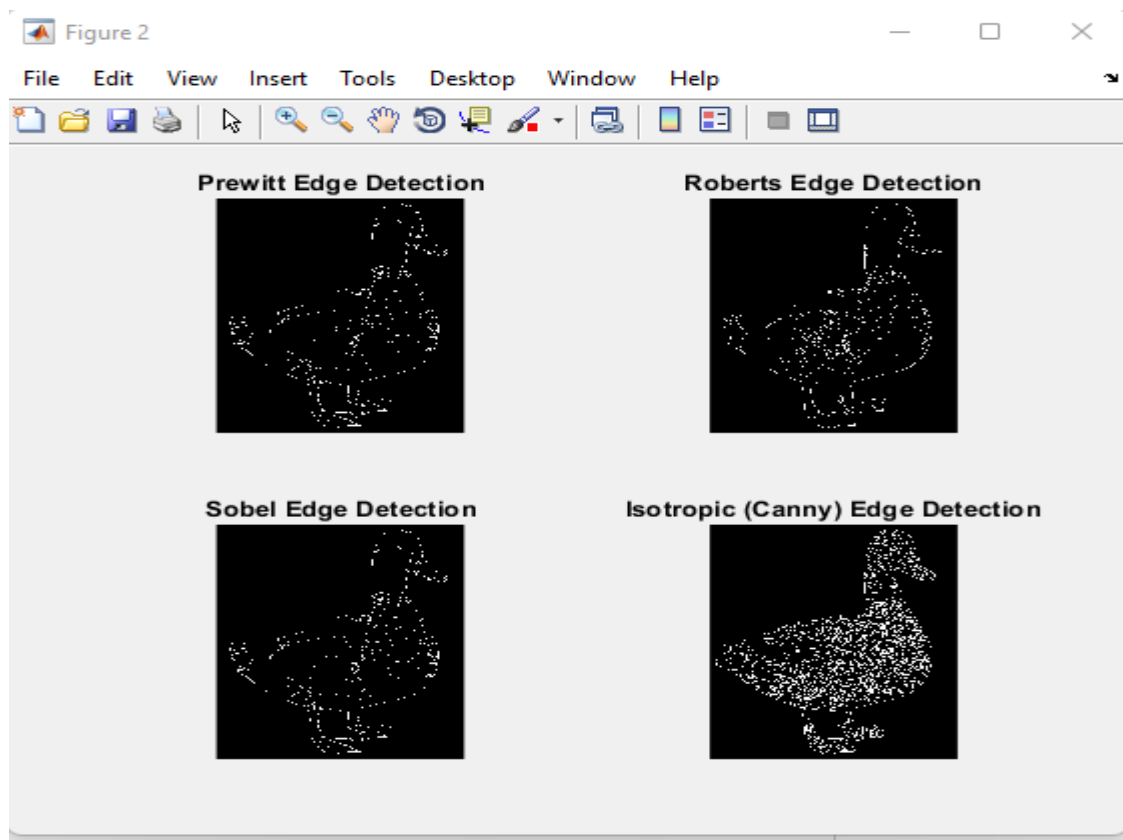
## Source Code:

```
% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\LAB05\EdgeDetection.jpeg');
gray_img = rgb2gray(img); % Convert to grayscale if it's an RGB image
% Prewitt operator
prewitt_edges = edge(gray_img, 'Prewitt');
% Roberts operator
roberts_edges = edge(gray_img, 'Roberts');
% Sobel operator
sobel_edges = edge(gray_img, 'Sobel');
% Isotropic operator (Approximation using Canny operator for isotropic behavior)
isotropic_edges = edge(gray_img, 'Canny');
% Displaying results
figure;
subplot(2,2,1), imshow(prewitt_edges), title('Prewitt Edge Detection');
subplot(2,2,2), imshow(roberts_edges), title('Roberts Edge Detection');
subplot(2,2,3), imshow(sobel_edges), title('Sobel Edge Detection');
subplot(2,2,4), imshow(isotropic_edges), title('Isotropic (Canny) Edge Detection');
```

## Input:



**Output:**



**Experiment Name:**

- Converting RGB image to HIS image.
- Converting RGB image to Grayscale image.
- Converting RGB image to Red, Green, Blue image.

**a. Objectives:**

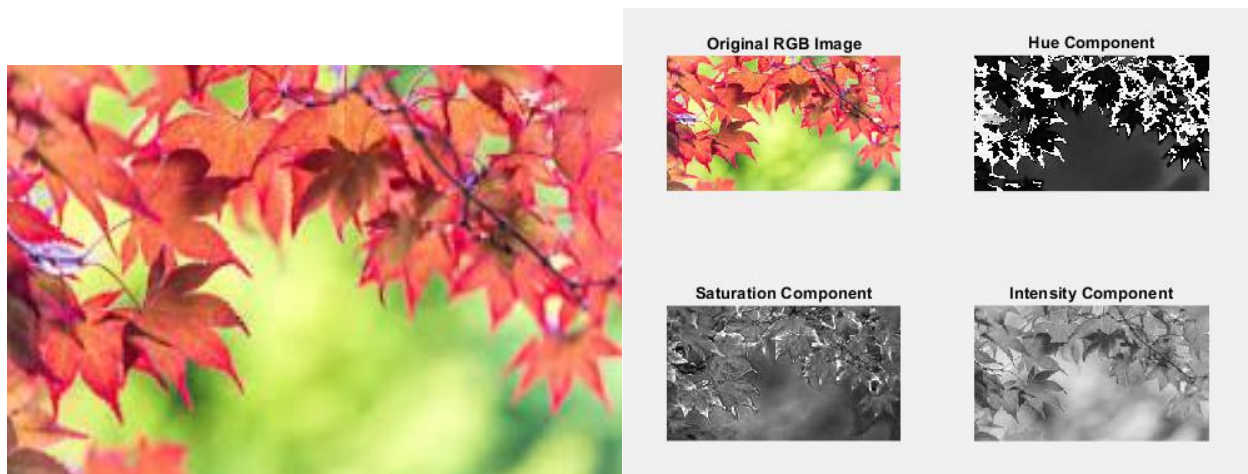


The objective of converting an RGB image to an HSI (Hue, Saturation, Intensity) image is to represent the image in a way that separates color information (Hue and Saturation) from intensity information (brightness), which can be beneficial for various image processing tasks.

### Source Code:

```
function [H, S, I] = rgb2hsi_image(input_path)
% Function to convert an RGB image to HSI
% Input: input_path - the file path of the input RGB image
% Output: H - the Hue component
%         S - the Saturation component
%         I - the Intensity component
% Read the RGB image from the input path
rgb_image = imread('C:\Users\CSELAB-2\Desktop\LAB06\InputPicture.jpg');
% Convert the RGB image to double for accurate calculations
rgb_image = im2double(rgb_image);
% Separate the RGB channels
R = rgb_image(:,:,1);
G = rgb_image(:,:,2);
B = rgb_image(:,:,3);
% Compute the intensity component (I)
I = (R + G + B) / 3;
% Compute the saturation component (S)
min_RGB = min(min(R, G), B);
S = 1 - (3 ./ (R + G + B + eps)) .* min_RGB;
% Compute the hue component (H)
theta = acos(0.5 * ((R - G) + (R - B)) ./ sqrt((R - G).^2 + (R - B) .* (G - B) + eps));
H = theta;
H(B > G) = 2 * pi - H(B > G);
H = H / (2 * pi); % Normalize to [0, 1]
% Display the original RGB image and H, S, I components
figure;
subplot(2,2,1);
imshow(rgb_image);
title('Original RGB Image');
subplot(2,2,2);
imshow(H);
title('Hue Component');
subplot(2,2,3);
imshow(S);
title('Saturation Component');
subplot(2,2,4);
imshow(I);
title('Intensity Component');
end
```

### Input and Output:



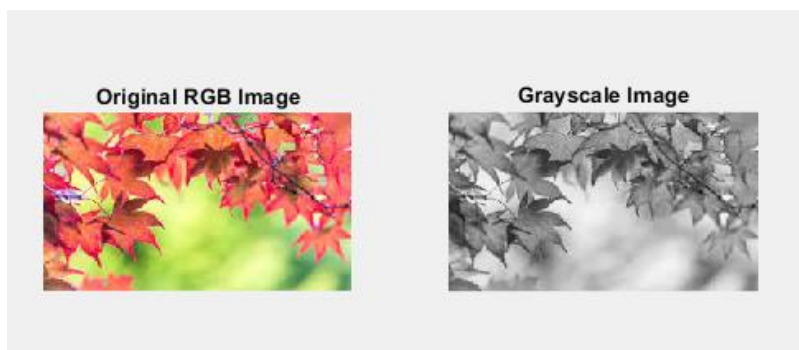
## b. Objectives:

The objective of converting an RGB image to a grayscale image is to reduce the image to its intensity values, eliminating color information while retaining structural and brightness details.

## Source Code:

```
function grayscale_image = rgb_to_grayscale(input_path)
% Function to convert an RGB image to grayscale
% Input: input_path - the file path of the input RGB image
% Output: grayscale_image - the grayscale image
% Read the RGB image from the input path
rgb_image = imread('C:\Users\CSELAB-2\Desktop\LAB06\InputPicture.jpg');
% Convert the RGB image to grayscale using the built-in function
grayscale_image = rgb2gray(rgb_image);
% Display the original RGB image and the grayscale image
figure;
subplot(1,2,1);
imshow(rgb_image);
title('Original RGB Image');
subplot(1,2,2);
imshow(grayscale_image);
title('Grayscale Image');
end
```

## Input and Output:



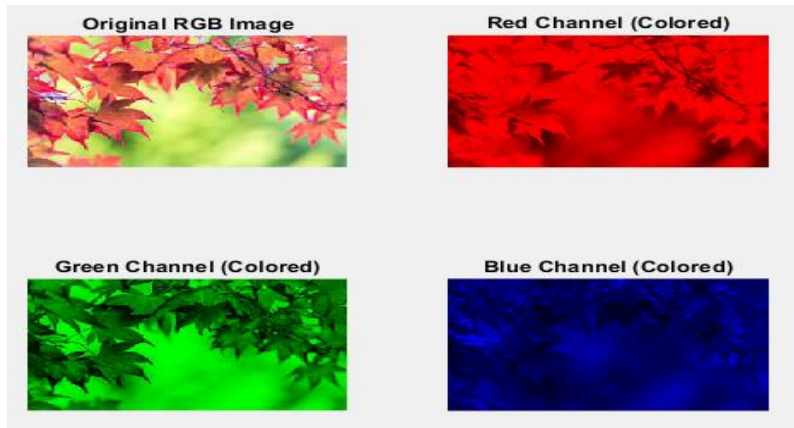
### c. Objectives:

The objective of converting an RGB image into separate Red, Green, and Blue channel images is to isolate and analyze the contribution of each individual color channel to the overall image.

### Source Code:

```
function [R_colored, G_colored, B_colored] = rgb_to_colored_channels(input_path)
    % Function to separate an RGB image into colored Red, Green, and Blue channels
    % Input: input_path - the file path of the input RGB image
    % Output: R_colored - Red channel as a colored image
    %         G_colored - Green channel as a colored image
    %         B_colored - Blue channel as a colored image
    % Read the RGB image from the input path
    rgb_image = imread('C:\Users\CSELAB-2\Desktop\LAB06\InputPicture.jpg');
    % Initialize colored channel images
    R_colored = zeros(size(rgb_image)); % Red channel image
    G_colored = zeros(size(rgb_image)); % Green channel image
    B_colored = zeros(size(rgb_image)); % Blue channel image
    % Assign the R, G, B values to the respective channels
    R_colored(:,1) = rgb_image(:,1); % Keep Red channel
    G_colored(:,2) = rgb_image(:,2); % Keep Green channel
    B_colored(:,3) = rgb_image(:,3); % Keep Blue channel
    % Convert channel images to uint8 for proper display
    R_colored = uint8(R_colored);
    G_colored = uint8(G_colored);
    B_colored = uint8(B_colored);
    % Display the original RGB image and colored R, G, B channel images
    figure;
    subplot(2,2,1);
    imshow(rgb_image);
    title('Original RGB Image');
    subplot(2,2,2);
    imshow(R_colored);
    title('Red Channel (Colored)');
    subplot(2,2,3);
    imshow(G_colored);
    title('Green Channel (Colored)');
    subplot(2,2,4);
    imshow(B_colored);
    title('Blue Channel (Colored)');
end
```

### Input and Output:



## Experiment Name:

- a. Dilation of an image.
- b. Erosion Operation of an image.
- c. Opening and Closing Operation of an image.
- d. FFT operation of an image.
- e. Homomorphic Transform of an image.

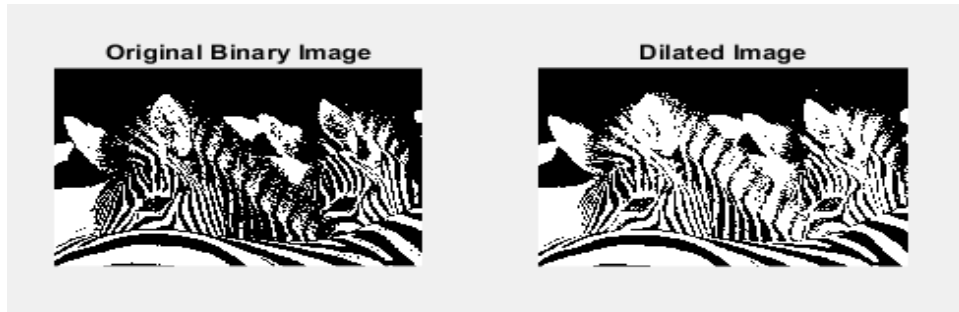
### a. Objectives:

The objectives of dilation in image processing include expanding the size of objects in a binary image to enhance their visibility, filling in small holes or gaps to reduce noise, connecting nearby objects for better recognition, and improving boundary detection. Dilation is a fundamental morphological operation that aids in feature enhancement and serves as a preprocessing step for tasks like segmentation and object recognition, making it essential for various applications, including medical imaging, character recognition, and structural analysis.

### Source Code:

```
% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\LAB07\Dia.jpg'); % Replace with your image file
% Convert to grayscale if it's a color image
if size(img, 3) == 3
    img = rgb2gray(img);
end
% Convert to binary image
bw = imbinarize(img);
% Define a structuring element
se = strel('disk', 5); % You can change the size and shape
% Perform dilation
dilated_image = imdilate(bw, se);
% Display the original and dilated images
figure;
subplot(1, 2, 1);
imshow(bw);
title('Original Binary Image');
subplot(1, 2, 2);
imshow(dilated_image);
title('Dilated Image');
```

## Input and Output:



## b. Objectives:

The objectives of erosion in image processing are to reduce the size of objects in a binary image, effectively removing small noise and isolated pixels, which helps in refining object boundaries and enhancing shape clarity. This operation aids in separating closely positioned objects, facilitating better feature extraction and analysis. Erosion serves as a foundational morphological operation that can be used in conjunction with dilation and other techniques, making it crucial for applications like medical imaging, industrial inspection, and character recognition, where precise object delineation is essential.

## Source Code:

```
% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\LAB07\Dia.jpg'); % Replace with your image file
% Convert to grayscale if it's a color image
if size(img, 3) == 3
    img = rgb2gray(img);
end
% Convert to binary image
bw = imbinarize(img);
% Define a structuring element
se = strel('disk', 5); % You can change the size and shape
% Perform erosion
eroded_image = imerode(bw, se);
% Display the original and eroded images
figure;
subplot(1, 2, 1);
imshow(bw);
title('Original Binary Image');
subplot(1, 2, 2);
imshow(eroded_image);
title('Eroded Image');
```

## Input and Output:



### c. Objectives:

The objectives of opening and closing operations in image processing are to refine and enhance the shapes of objects within a binary image. Opening, which combines erosion followed by dilation, is used to remove small noise and separate connected objects, helping to smooth object boundaries and eliminate small artifacts. Closing, on the other hand, involves dilation followed by erosion, and is effective for filling small holes and gaps within objects, thereby enhancing their connectivity. Together, these morphological operations facilitate better image analysis, segmentation, and feature extraction in applications such as medical imaging, object recognition, and industrial inspection, where accurate representation of shapes is crucial.

### Source Code:

```
% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\LAB07\Dia.jpg'); % Replace with your image file
% Convert to grayscale if it's a color image
if size(img, 3) == 3
    img = rgb2gray(img);
end
% Convert to binary image
bw = imbinarize(img);
% Define a structuring element
se = strel('disk', 5); % You can change the size and shape
% Perform opening
opened_image = imopen(bw, se);
% Perform closing
closed_image = imclose(bw, se);
% Display the original and processed images
figure;
subplot(1, 3, 1);
imshow(bw);
title('Original Binary Image');
subplot(1, 3, 2);
imshow(opened_image);
title('Opened Image');
subplot(1, 3, 3);
imshow(closed_image);
title('Closed Image');
```

### Input and Output:



### d. Objectives:

The objectives of performing a Fast Fourier Transform (FFT) on an image are to analyze its frequency components, enabling the transformation of spatial information into the frequency domain. This allows for various applications such as filtering, compression, and image enhancement. By decomposing an image into its frequency components, FFT facilitates the identification of patterns, noise reduction, and the application of frequency-based filters (e.g., low-pass and high-pass filters) to manipulate specific features. Ultimately, FFT is a powerful tool for improving image quality, enabling efficient processing, and aiding in tasks like feature extraction and image reconstruction in diverse fields such as medical imaging, telecommunications, and computer vision.

### Source Code:

```
% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\LAB07\Dia.jpg'); % Replace with your image file
% Convert to grayscale if it's a color image
if size(img, 3) == 3
    img = rgb2gray(img);
end
% Convert to double for FFT processing
img = double(img);
% Compute the FFT of the image
fft_image = fft2(img);
% Shift the zero-frequency component to the center
fft_shifted = fftshift(fft_image);
% Compute the magnitude spectrum
magnitude_spectrum = log(1 + abs(fft_shifted));
% Display the original image and its FFT magnitude spectrum
figure;
subplot(1, 2, 1);
imshow(img, []);
title('Original Image');
subplot(1, 2, 2);
imshow(magnitude_spectrum, []);
title('Magnitude Spectrum (FFT)');
```

### Input and Output:



### e. Objectives:

The objectives of applying a homomorphic transform to an image are to enhance its contrast and improve visibility by simultaneously manipulating its illumination and reflectance components. This process transforms the image into the frequency domain, allowing for the separation of low-frequency (illumination) and high-frequency (reflectance) information. By applying appropriate filters, homomorphic transforms can reduce the effects of uneven lighting and enhance fine details, resulting in improved image quality and clarity. This technique is particularly useful in applications such as medical imaging, remote sensing, and object recognition, where consistent illumination is crucial for accurate analysis and interpretation.

### Source Code:

```
% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\LAB07\Dia.jpg'); % Replace with your image file
% Convert to grayscale if it's a color image
if size(img, 3) == 3
    img = rgb2gray(img);
end
% Convert to double for processing
img = double(img) / 255; % Normalize the image
% Parameters for the homomorphic transform
gammaL = 0.3; % Low frequency gain
gammaH = 2.0; % High frequency gain
```

```

cutoff = 30; % Cutoff frequency for the filter
% Compute the FFT of the image
fft_image = fft2(img);
fft_shifted = fftshift(fft_image);
% Create a Gaussian filter
[M, N] = size(img);
[x, y] = meshgrid(-N/2:N/2-1, -M/2:M/2-1);
D = sqrt(x.^2 + y.^2);
H = 1 - exp(-(D.^2) / (2 * (cutoff^2))); % Homomorphic filter
% Apply the filter to the shifted FFT
filtered_fft = H .* fft_shifted;
% Inverse FFT to get back to the spatial domain
inverse_fft = ifft2(ifftshift(filtered_fft));
enhanced_img = abs(inverse_fft);
% Apply gamma correction
enhanced_img = (gammaH * enhanced_img) ./ (gammaL * (1 - enhanced_img));
% Normalize the enhanced image
enhanced_img = mat2gray(enhanced_img);
% Display the original and enhanced images
figure;
subplot(1, 2, 1);
imshow(img, []);
title('Original Image');
subplot(1, 2, 2);
imshow(enhanced_img, []);
title('Enhanced Image (Homomorphic Transform)');

```

### Input and Output:



### Experiment Name:

- How will an image look after adding Gaussian, Rayleigh and Erlang Noise.
- Image corruption by Gaussian Noise after 3\*3 Arithmetic and Geometric Mean Filter.
- Removal of Salt and Pepper Noise.
- Periodic Noise Removal.

### a. Objectives:

To demonstrate how different types of noise affect an image.

- Gaussian Noise:** This noise appears as random variations in brightness across the image, causing a grainy effect.



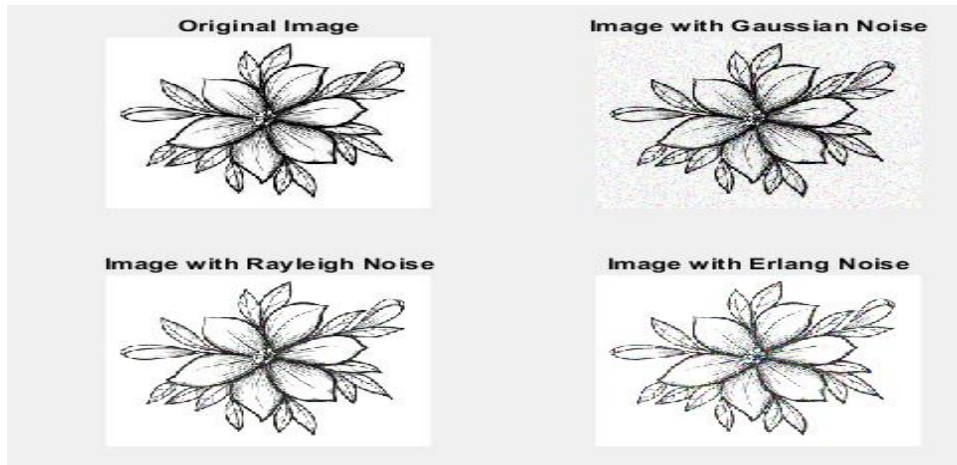
- **Rayleigh Noise:** Typically appears in images with a lot of low-intensity details, leading to random bright spots and variations.
- **Erlang Noise:** Similar to Gaussian but has a heavier tail, often making it more prominent in certain regions.

After adding these noises, the image will appear distorted, with variations in pixel intensity that degrade the overall quality.

### Source Code:

```
% Read an image
img = imread('C:\Users\LAB203\Desktop\Input.jpg'); % Replace with your image file
img = im2double(img); % Convert to double for processing
% Add Gaussian Noise
mean_g = 0; % Mean of Gaussian noise
var_g = 0.01; % Variance of Gaussian noise
gaussian_noise = mean_g + sqrt(var_g) * randn(size(img));
img_gaussian = img + gaussian_noise;
% Add Rayleigh Noise
scale_r = 0.1; % Scale parameter for Rayleigh noise
rayleigh_noise = raylrnd(scale_r, size(img));
img_rayleigh = img + rayleigh_noise;
% Add Erlang Noise
k = 2; % Shape parameter (k > 0)
scale_e = 0.1; % Scale parameter for Erlang noise
erlang_noise = gamrnd(k, scale_e, size(img)); % Gamma distribution for Erlang
img_erlang = img + erlang_noise;
% Ensure that the images remain in the range [0, 1] after adding noise
img_gaussian = min(max(img_gaussian, 0), 1);
img_rayleigh = min(max(img_rayleigh, 0), 1);
img_erlang = min(max(img_erlang, 0), 1);
% Display images in one figure
figure;
% Original Image
subplot(2, 2, 1); % 2 rows, 2 columns, 1st subplot
imshow(img);
title('Original Image');
% Image with Gaussian Noise
subplot(2, 2, 2); % 2 rows, 2 columns, 2nd subplot
imshow(img_gaussian);
title('Image with Gaussian Noise');
% Image with Rayleigh Noise
subplot(2, 2, 3); % 2 rows, 2 columns, 3rd subplot
imshow(img_rayleigh);
title('Image with Rayleigh Noise');
% Image with Erlang Noise
subplot(2, 2, 4); % 2 rows, 2 columns, 4th subplot
imshow(img_erlang);
title('Image with Erlang Noise');
```

### Input and Output:



## b. Objectives:

To show the effect of filtering techniques after adding noise.

- **Gaussian Noise Addition:** The image becomes grainy.
- **Arithmetic Mean Filter:** This filter averages pixel values in a  $3 \times 3$  neighborhood. The result is a smoothed image but may lose finer details.
- **Geometric Mean Filter:** This filter preserves edges better than the arithmetic mean filter, resulting in a smoother image while retaining more detail.

After applying both filters, the image will appear less noisy, with the arithmetic mean resulting in a more blurred effect compared to the more detailed preservation from the geometric mean.

## Source Code:

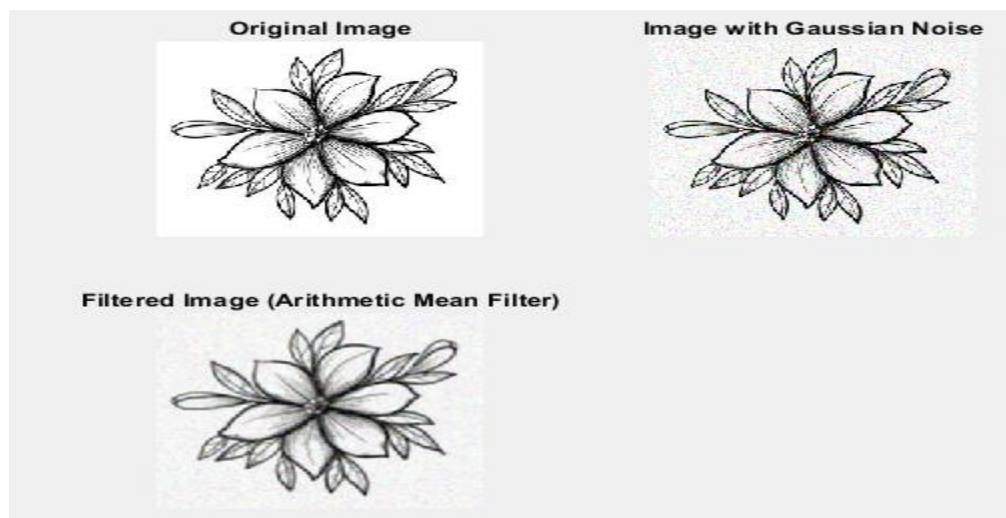
```
% Read an image
img = imread('C:\Users\CSELAB-2\Desktop\Labb8\Input.jpg'); % Replace with your image file
img = im2double(img); % Convert to double for processing
% Display original image
figure;
% Original Image
subplot(2, 2, 1); % 2 rows, 2 columns, 1st subplot
imshow(img);
title('Original Image');
% Add Gaussian Noise
mean_g = 0; % Mean of Gaussian noise
var_g = 0.01; % Variance of Gaussian noise
gaussian_noise = mean_g + sqrt(var_g) * randn(size(img)); % Generate Gaussian noise
noisy_img = img + gaussian_noise; % Corrupt the image with noise
% Ensure the noisy image remains within the valid range [0, 1]
noisy_img = min(max(noisy_img, 0), 1);
% Display image with Gaussian Noise
subplot(2, 2, 2); % 2nd subplot
imshow(noisy_img);
title('Image with Gaussian Noise');
% Apply 3x3 Arithmetic Mean Filter
mean_filter = fspecial('average', 3); % Create a 3x3 mean filter
filtered_mean_img = imfilter(noisy_img, mean_filter, 'replicate'); % Apply filter
% Display filtered image using Arithmetic Mean Filter
subplot(2, 2, 3); % 3rd subplot
```

```

imshow(filtered_mean_img);
title('Filtered Image (Arithmetic Mean Filter)');
% Apply 3x3 Geometric Mean Filter
% Initialize the filtered image
filtered_geom_img = zeros(size(noisy_img));
% Apply Geometric Mean Filter
for i = 1:size(noisy_img, 1)
    for j = 1:size(noisy_img, 2)
        % Get the neighborhood
        neighborhood = noisy_img(max(i-1, 1):min(i+1, size(noisy_img, 1)), ...
            max(j-1, 1):min(j+1, size(noisy_img, 2)), :);
        % Calculate the geometric mean for each channel
        filtered_geom_img(i, j, :) = exp(mean(log(neighborhood + eps), [1 2])); % +eps to avoid log(0)
    end
end
% Display filtered image using Geometric Mean Filter
subplot(2, 2, 4); % 4th subplot
imshow(filtered_geom_img);
title('Filtered Image (Geometric Mean Filter)');

```

### Input and Output:



### c. Objectives:

To demonstrate noise reduction techniques for salt and pepper noise.

- **Salt and Pepper Noise:** This noise manifests as randomly occurring white and black pixels.
- **Removal Technique:** Typically achieved using a median filter, which effectively removes this type of noise by replacing each pixel with the median value of its neighbors.

After applying the median filter, the image should look cleaner, with significantly fewer noisy pixels.

### Source Code:

```

% Read an image
img = imread('C:\Users\LAB203\Desktop\Input.jpg'); % Replace with your image file

```

```

img = im2double(img); % Convert to double for processing
% Add Salt and Pepper Noise
noise_density = 0.05; % Density of noise
noisy_img = imnoise(img, 'salt & pepper', noise_density);
% Remove Salt and Pepper Noise using Median Filter
filtered_img = img; % Initialize filtered image
if size(noisy_img, 3) == 3
    % If the image is RGB, apply median filter to each channel
    filtered_img = zeros(size(noisy_img)); % Initialize filtered image for RGB
    for i = 1:3
        filtered_img(:, :, i) = medfilt2(noisy_img(:, :, i)); % Apply median filter to each channel
    end
else
    % For grayscale image
    filtered_img = medfilt2(noisy_img);
end
% Display images in one figure
figure;
% Original Image
subplot(1, 3, 1); % 1 row, 3 columns, 1st subplot
imshow(img);
title('Original Image');
% Image with Salt and Pepper Noise
subplot(1, 3, 2); % 1 row, 3 columns, 2nd subplot
imshow(noisy_img);
title('Image with Salt and Pepper Noise');
% Filtered Image
subplot(1, 3, 3); % 1 row, 3 columns, 3rd subplot
imshow(filtered_img);
title('Filtered Image (Median Filter)');

```

### Input and Output:



**d. Objectives:** To illustrate the process of removing periodic noise using frequency domain techniques.

- **Periodic Noise:** Appears as regular patterns or stripes in an image.
- **Removal Technique:** The process involves using the Fourier Transform to identify and filter out the periodic components.

After processing, the image should show reduced regular patterns, restoring the visual integrity of the original image.

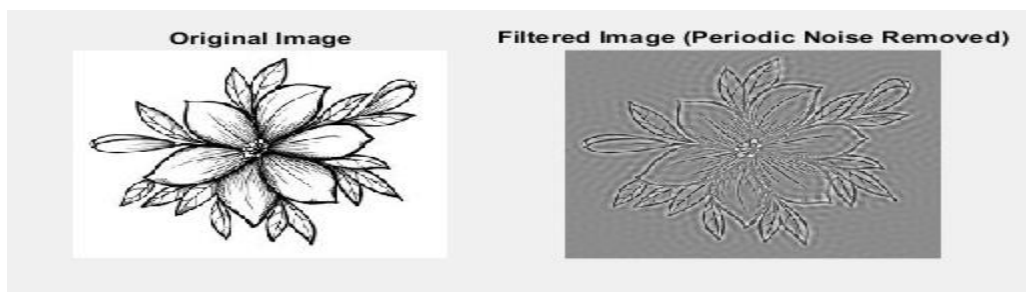
### Source Code:

```

% Read an image
img = imread('C:\Users\CSELAB-2\Desktop\Labb8\Input.jpg'); % Replace with your image file
img = im2double(img); % Convert to double for processing
% Convert to grayscale if the image is RGB
if size(img, 3) == 3
    img_gray = rgb2gray(img);
else
    img_gray = img;
end
% Perform FFT
F = fft2(img_gray);
F_shifted = fftshift(F); % Shift zero frequency components to center
% Create a filter to remove periodic noise
[m, n] = size(img_gray);
cutoff_radius = 30; % Adjust as necessary for your image
[x, y] = meshgrid(-n/2:n/2-1, -m/2:m/2-1);
distance = sqrt(x.^2 + y.^2);
% Create a circular high-pass filter
hp_filter = double(distance > cutoff_radius);
% Apply the filter in the frequency domain
F_filtered = F_shifted .* hp_filter;
% Inverse FFT to get the filtered image
filtered_F = ifftshift(F_filtered); % Shift back
img_filtered = ifft2(filtered_F);
img_filtered = real(img_filtered); % Take the real part
% Display input and output images in one figure
figure;
% Input Image
subplot(1, 2, 1); % 1 row, 2 columns, 1st subplot
imshow(img_gray);
title('Original Image');
% Filtered Image
subplot(1, 2, 2); % 1 row, 2 columns, 2nd subplot
imshow(img_filtered, []);
title('Filtered Image (Periodic Noise Removed)');

```

### Input and Output:



### Experiment Name:

- a. Hough Transformation for Straight Line Detection.
- b. Hough Transformation for Circle Detection.

### a. Objectives:

The objective of the Hough Transform for straight line detection is to accurately identify and localize straight lines within an image, even in noisy, cluttered, or low-contrast environments. This approach is robust against partial occlusions and discontinuities, allowing for the detection of multiple lines with varying orientations. By accumulating evidence in parameter space, the Hough Transform enables efficient and precise line detection, making it valuable for applications such as computer vision, robotic navigation, and automated analysis in fields where accurate and adaptable line detection is essential.

### Source Code:

```
% Read an image
img = imread('C:\Users\CSELAB-2\Desktop\Lab09\Input.jpg'); % Replace with your image file path
gray_img = rgb2gray(img); % Convert to grayscale
% Apply edge detection
edges = edge(gray_img, 'canny'); % Use Canny edge detector
% Apply the Hough Transform
[H, theta, rho] = hough(edges);
% Detect peaks in the Hough Transform
peaks = houghpeaks(H, 5, 'threshold', ceil(0.3 * max(H(:))));
% Find lines based on Hough peaks
lines = houghlines(edges, theta, rho, peaks, 'FillGap', 5, 'MinLength', 7);
% Display the results
figure, imshow(img), hold on
for k = 1:length(lines)
    % Get endpoints of each line segment
    xy = [lines(k).point1; lines(k).point2];
    % Plot each line in red
    plot(xy(:,1), xy(:,2), 'LineWidth', 2, 'Color', 'red');
    % Mark the beginnings and ends of each line with a green circle
    plot(xy(1,1), xy(1,2), 'x', 'LineWidth', 2, 'Color', 'green');
    plot(xy(2,1), xy(2,2), 'x', 'LineWidth', 2, 'Color', 'yellow');
end
title('Detected Lines')
hold off
```

### Input and Output:



### b. Objectives:

The objective of the Hough Transform for circle detection is to accurately and efficiently identify and localize circular shapes within an image by determining their centers and radii, even in noisy or cluttered backgrounds. This method enables robust detection across varying circle sizes, partial occlusions, and image quality variations, making it highly applicable in fields like medical imaging, industrial inspection, and automated object recognition where precise, adaptable, and automated circle detection is essential.

### Source Code:

```

% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\Lab09\Input.jpg'); % Replace with your image file path
gray_img = rgb2gray(img); % Convert to grayscale if necessary
% Define the radius range for the circles (adjust based on your image)
min_radius = 20;
max_radius = 50;
% Perform Hough Transform to find circles
[centers, radii, metric] = imfindcircles(gray_img, [min_radius max_radius], ... 'Sensitivity', 0.9, 'EdgeThreshold', 0.1);
% Display the original image
figure, imshow(img), hold on
% Plot detected circles
viscircles(centers, radii, 'EdgeColor', 'b');
% Plot the centers of the circles
plot(centers(:,1), centers(:,2), 'x', 'LineWidth', 2, 'Color', 'r');
title('Detected Circles');
hold off

```

### Input and Output:

